

A LegalBERT-Based Approach for Extractive Text Summarization of Legal Documents

Report submitted to the SASTRA Deemed to be University as the requirement for the course

CSE300 – MINI PROJECT

Submitted by

Dhivya Saki M (Reg. No. 227003041, BTech CSE)

Meagha S (Reg. No. 227003090, BTech CSE)

Shaik Asifa (Reg. No. 227003131, BTech CSE)

May 2026



**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
SRINIVASA RAMANUJAN CENTRE
KUMBAKONAM, TAMIL NADU,
INDIA – 612001**



THINK MERIT | THINK TRANSPARENCY | THINK SASTRA

**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
SRINIVASA RAMANUJAN CENTRE
KUMBAKONAM, TAMIL NADU,
INDIA – 612001**

Bonafide Certificate

This is to certify that the project report titled “**A LegalBERT-Based Approach for Extractive Text Summarization of Legal Documents**” submitted as a requirement for the course, CSE300 – MINI PROJECT for the degree of **B.Tech Computer Science and Engineering** at SASTRA Deemed to be University, is a bona-fide record of the work done by **Ms. Dhivyasaki M** (Reg. No. 227003041), **Ms. Meaghaa S** (Reg. No. 227003090), **Ms. Shaik Asifa** (Reg. No. 227003131) during the sixth semester of the academic year 2025-26, in the Department of Computer Science and Engineering, Srinivasa Ramanujan Centre, under my supervision. This report has not formed for the basis for the award of any degree, diploma, associateship, fellowship or other similar title to any candidate of any University.

Signature of Project Supervisor:

Name with Affiliation : **Smt. S. Hemamalini AP-II/CSE**

Date :

Project *Viva voce* held on _____

Examiner 1

Examiner 2



THINK MERIT I THINK TRANSPARENCY I THINK SASTRA

**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
SRINIVASA RAMANUJAN CENTRE
KUMBAKONAM, TAMIL NADU,
INDIA – 612001**

Declaration

We declare that the project report titled “**A LegalBERT-Based Approach for Extractive Text Summarization of Legal Documents**” submitted by us is an original work done by us under the guidance of **Smt. S. Hemamalini AP-II/CSE, Srinivasa Ramanujan Centre, SASTRA Deemed to be University** during the sixth semester of the academic year 2025-26, in the Srinivasa Ramanujam Centre. The work is original and wherever we have used materials from other sources, we have given due credit and cited them in the text of the report. This report has not formed the basis for the award of any degree, diploma, associate-ship, fellowship or other similar title to any candidate of any University.

Name of the Candidates

Signature of the Candidates

Dhivyasaki M

Meaghaa S

Shaik Asifa

Date:

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our Honorable Chancellor **Prof. R. Sethuraman**, for providing us with the opportunity and the necessary infrastructure to carry out this project as part of our curriculum.

We would like to thank our Honorable Vice-Chancellor, **Dr. S. Vaidhyasubramaniam**, and **Dr. S. Swaminathan**, Dean of Planning & Development, for their constant encouragement and strategic support throughout our college journey. We also extend our sincere gratitude to **Dr. R. Chandramouli**, Registrar, SASTRA Deemed to be University, for providing us with the opportunity to pursue this project.

We extend our heartfelt thanks to **Dr. B. Santhi**, Dean, **Dr. A. Alli Rani**, Associate Dean, **Dr. D. Narasimhan**, Associate Dean, Srinivasa Ramanujan Centre, SASTRA Deemed to be University. We exhibit our pleasure in expressing our thanks to **Dr. V. Kalaichelvi**, Associate Professor, Department of Computer Science and Engineering, Srinivasa Ramanujan Centre, for her encouragement during our project work.

Our guide **Smt. S. Hemamalini**, Assistant Professor - II, Department of Computer Science and Engineering, Srinivasa Ramanujan Centre was the driving force behind this whole idea from the start. Her deep insight in the field and invaluable suggestions helped us in making progress throughout our project work.

We also thank the project review panel members **Dr. P. Umamaheswari**, Assistant Professor - III and **Smt. M. Anuradha**, Assistant Professor - I, for their valuable comments and insights which made this project better.

We would like to extend our gratitude to **Dr. N. Moratanch**, Assistant Professor - III & Project Co-ordinator and **Dr. S. Suganya**, Assistant Professor - II & Project Co-ordinator, Department of Computer Science and Engineering, Srinivasa Ramanujan Centre for organizing and helping us in the completion of the project.

We gratefully acknowledge all the contributions and encouragement from my family and friends resulting in the successful completion of this project. We thank you all for giving us the opportunity to showcase our skills through this project.

List of Figures

Figure No	Title	Page No
Figure 1	Proposed Framework for Legal Document Summarization	4

List of Tables

Table No	Title	Page No
Table 1	Dataset Statistics	7
Table 2	Supervised Results (ROUGE)	9
Table 3	Unsupervised Results	10

Abbreviations

BERT	Bidirectional Encoder Representations from Transformers
LegalBERT	BERT model trained on legal domain text
LLM	Large Language Model
NLP	Natural Language Processing
TR	TextRank
MMR	Maximal Marginal Relevance
LDA	Latent Dirichlet Allocation
KMeans	K-Means Clustering
ROUGE	Recall Oriented Understudy for Gisting Evaluation
P	Precision
R	Recall
F1	F1 Score
Top-K	Top K evaluation metric
TF-IDF	Term Frequency – Inverse Document Frequency
CSV	Comma Separated Values
JSON	JavaScript Object Notation
NLTK	Natural Language ToolKit
TQDM	Progress bar library

ABSTRACT

Extractive summarization of legal judgments aims to condense lengthy court documents into concise and meaningful summaries while preserving legal accuracy and original wording. Manual summarization is time-consuming and subjective, while rapid digital growth increases the need for automated assistance. Most existing systems mainly depend on basic ranking methods or manually labelled roles, and many advanced models do not properly consider the structure of legal documents. As a result, important sections of judgments may be missed, while less relevant parts, such as repeated legal references, are sometimes given too much importance. Therefore, a new system is needed to combine legal structure awareness with learned sentence importance and constrained selection to improve legal relevance, coverage, coherence, and redundancy reduction. This work proposes an extractive summarization framework that leverages LegalBERT for domain-specific contextual sentence representation and applies a Knapsack-based optimization strategy for effective and length-constrained sentence selection to capture judicial structure and legal significance. The effectiveness of the proposed framework will be evaluated using supervised metrics such as ROUGE, while unsupervised evaluation is performed using Kendall’s Tau and Spearman’s Correlation to measure ranking consistency and agreement. Its performance will be compared with baseline extractive summarization approaches to demonstrate improvements in summary quality and legal relevance.

Keywords: Extractive Summarization, Legal Judgments, LegalBERT, Knapsack Optimization, ROUGE and Rank Correlation Evaluation

TABLE OF CONTENT

Title	Page No
Bonafide Certificate	i
Declaration	ii
Acknowledgement	iii
List of Figures	iv
List of Tables	iv
Abbreviations	v
Abstract	vi
1. Summary of Base paper	1
1.1. Introduction	1
1.2. Problem Statement	1
1.3. Literature Review	1
1.4. Architecture Diagram	4
1.5. Software and Hardware Requirements	4
1.5.1. Software Requirements	4
1.5.2. Hardware Requirements	6
1.6. Modules and Description	7
1.6.1. Dataset Description	7
1.6.2. Preprocessing	8
1.6.3. Sematic Sentence Representation	8
1.6.4. Graph-based Sentence Ranking	8
1.6.5. Redundancy Removal and Diversity Enhancement	8
1.6.6. Summary Generation	9
1.6.7. Results and Discussion	9
1.6.7.1. Supervised Result (ROUGE)	9
1.6.7.2. Unsupervised Result	10
2. Merits and Demerits of Base paper	11
2.1. Merits	11
2.2. Demerits	12
3. Source Code and Results	13
3.1. Summarization	13
3.2. Evaluation	15
4. Conclusion and Future Plans	18
4.1. Conclusion	18
4.2. Future Plans	18
5. Reference	19

CHAPTER 1

1 SUMMARY OF BASE PAPER

1.1 Introduction

Legal verdicts may be long for their complicated nature. With the rise in volume of digitized data, manual reading of such materials consumes a lot of time. Information may become inaccessible due to difficult language used in the legal documents. Therefore, to solve this problem, the necessary sections in legal documents are selected while preserving the context of the document. This project entails the development of a system that generates clear and concise summaries. Clearness and understanding are important, as they enable people to understand the information better. Poor summaries arise from the failure of key information to show up clearly.

1.2 Problem Statement

Legal texts are intricate, extensive, and hard to comprehend for someone unfamiliar with the law. The conventional summarization techniques such as TF-IDF and TextRank are based only on shallow textual features and do not consider the deep semantics involved in legal language. Moreover, these techniques perceive judgments simply as raw text and disregard its structural significance. In order to address these issues, this system makes use of LegalBERT to create legal-domain semantic representations for sentences. It further integrates TextRank and MMR with semantic similarity and clustering.

1.3 Literature Review

The growing amount of legal information available in digital form has increased the demand for efficient automated document summarization systems. This is because the complexity associated with their long length, elaborate structures, and complex language poses difficulties in producing useful summaries manually. In response to this problem, various computational methods have been investigated. Most research conducted in this field focuses on extractive summarization methods.

An example of one such approach is the one described in [1]. It involves the use of an unsupervised method using clustering algorithms and knapsack optimization in order to

produce summaries. Though this does not require the availability of a labelled dataset, it is heavily reliant on the results of the clustering produced and fails to understand the semantics of the legal texts.

Another similar method proposed in [2] identifies important sentences in a document based on manually defined attributes such as the term frequency-inverse document frequency (TF-IDF), position in the document, and occurrence of keywords. Even though this approach works well enough, it cannot be used without a labelled dataset and consideration of contexts.

This gap is tackled in [3], where an ensemble approach was proposed, which combines different extractive summarization systems to generate better performance. The idea is to exploit the strengths of each system and integrate them into one. This leads to the lack of a standard ranking process while increasing the computational burden.

Another approach to tackle the problem is through the application of Integer Linear Programming for summarization. As mentioned in [4], the method is used to maintain balance between legal arguments and rhetoric using domain knowledge. Though this method is very powerful, its main drawback is that it is computationally expensive, making it difficult to apply widely.

Taking advantage of recent advances in deep learning technology, a new approach is introduced in [5], which selects sentences using a term-based approach through a diffusion process. Though interesting and promising, the algorithm is extremely complex and difficult to implement.

Additionally, another approach uses the hierarchy of documents by analysing the relationship between different layers, such as words, sentences, and paragraphs, in [6]. The main drawback of this approach is that it lacks contextual information since it relies only on syntactic parsing of the document.

The study conducted in [7] states that summarization has been done using a binary classification scheme, whereby sentences have been classified either as important or unimportant. Although it has been quite simple in implementation, it has been producing highly fragmented and inconsistent results since it fails to consider the level of significance and dependencies among the sentences.

The modern approaches to achieving highly optimized results involve the use of mixed strategies that involve not only extraction but also abstraction. This is shown in [8] where multiple transformer models have been used to extract and then refine sentences for creating accurate and clear summaries, although this comes with increased complexity.

Similarly, [9] employs a two-stage process that involves identifying key sentences followed by their rewriting to create more elegant outputs. Though it may take longer and become more complicated, the summaries produced become coherent and suitable for long legal documents. Moreover, in [10], transfer learning is applied, whereby transformer models are used to refine sentences and create summaries. Although this involves various levels of processing that increase complexity, it becomes effective even with limited training data.

Along similar lines, [11] leverages the structure of legal documents by extracting important parts before producing the summary output. While this renders the model extremely complicated due to its domain-specific nature and procedure, the generated summaries are highly structured and readable for the audience.

Moreover, [12] compares different approaches to generate summaries in terms of performance, ranging from abstractive approaches like BART and Legal Pegasus to extractive approaches such as LexRank and TextRank. While it is an informative comparative study about the mentioned models, it does not offer a solution in terms of a novel methodological approach.

Finally, [13] combines Transformer-based models with extractive methods, namely TF-IDF and TextRank, to generate effective summaries. Even though it enhances the quality of the process, it renders it complex from a technical perspective.

In yet another instance of an extractive method, [14] uses the graph-based entropy measure to determine the most relevant sentences and then utilizes language models for further filtering. This improves both the summarization results and the speed of processing. However, it requires performing several more calculations. Increasing attention is paid by scholars to the usage of large language models within legal areas.

For example, one research paper deals with many aspects of prompting techniques used when working with different types of LLMs in prompt engineering, IR, and fine-tuning [15]. Although this work addresses both extractive and abstractive summarization methodologies, it avoids favouring any specific LLM and focuses instead on methodologies and problems related to using LLMs in the legal domain.

It can be concluded that despite significant advances in the field of document summarization in law, there are still numerous challenges faced by researchers, namely redundancy, insufficient semantic understanding, and inappropriate sentence selection. In order to deal with these issues, the suggested framework for extractive domain-specific summarization includes LegalBERT as the context-aware encoder, Semantic TextRank as the technique of accurate sentence extraction, KMeans as the way to diversify topics and MMR to reduce redundancy.

1.4 Architecture Diagram

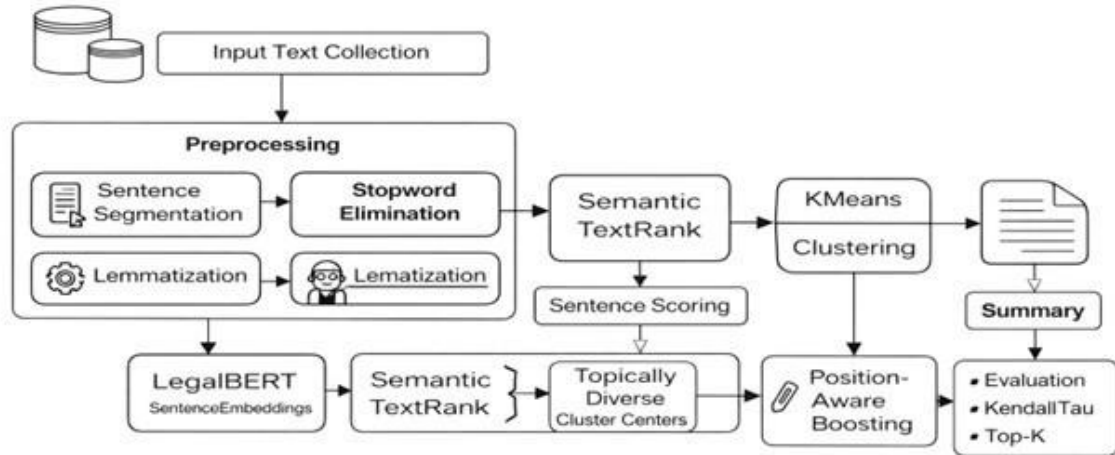


Figure 1: Proposed Framework for Legal Document Summarization

1.5 Software and Hardware Requirements

1.5.1 Software Requirements

The developed software application which reduces the length of texts legally is built based on the Python programming language, operating in Google Colab to easily deal with large amounts of data. Heavy computation operations depend on this cloud-based program, since its machine learning part will operate information without any lags. Cloud computing helps perform complex computational tasks one after another. All stages including data preprocessing, data evaluation, sorting and grouping of objects can be easily performed with the use of Python language. The transition of phases starts with initial preparation and ends with an analysis. It goes through all the necessary procedures and includes not only processing but also assessment.

- **Google Colab**

Working with all phases from the start is convenient, especially considering the usage of GPU along with Google Drive integration.

- **PyTorch**

Loading the legal-bert model with further handling of the information flow. Operations with tensors are executed here automatically.

Transformers by Hugging Face

Used to load the pretrained LegalBERT model (nlpaueb/legal-bert-base-uncased).

- **NLTK**

Sentences have to be split up prior to any cleaning process since this occurs early to allow for future separation into parts.

- **Scikit-learn**

Cosine similarity calculation measures the likeness between sentences. Further grouping relies on KMeans sorting similar objects. The latter procedure follows after, but both remain independent from one another.

- **NumPy and Pandas**

Responsible for handling numerical calculations and processing data in CSV files.

Although performing computations, it also performs file operations effectively.

- **ROUGE-Score library**

Provides scoring of summaries by comparing them against references in terms of ROUGE-1 scores. One approach includes the measurement of the overlap in terms of structurally similar words. Alternatively, ROUGE-2 calculates overlap of neighbouring word pairs. There is also an option to pay attention to long matches of sequences found across outputs using ROUGE-L.

- **TQDM**

Library used for creating progress bars while processing many documents. JSON and Operating System Libraries Interim results are kept in this library while being processed. Progress states remain available in case of an emergency interruption.

1.5.2 Hardware Requirements

Such high computational resources become necessary due to the large number of texts that need to be processed in transformer models. It is caused by the rapid increase in model scalability along with the growing amount of data.

- **RAM**

Better results can be obtained starting from 8 GB and with a rather large set of information consisting of about 5 thousand files to operate on efficiently. Splitting the task into pieces will be easier with the required amount of memory.

- **Processor**

A good processor like Intel i5 or i7 makes processing faster. Although the number of cores increases the speed, similar processors produce the same results. There are some alternative processors that perform equally well under heavy loads.

- **GPU (Suggested)**

An NVIDIA GPU which can process CUDA code - such as the Tesla T4 used in Google Colab - accelerates processes involving the usage of LegalBERT and generating batches of embeddings. Even though the device is intended for intensive computations, it is highly efficient for performing the particular operations thanks to its inherent abilities. It is possible to utilize the system using a computer's CPU.

However, it will take significantly more time to complete each action.

- **Storage**

Around 10 to 20 gigabytes of free space on the hard disk are necessary for storing collections of data and summary files, which are saved as CSV or JSON files.

- **Cloud Storage**

Data images are stored in the cloud storage provided by Google Drive, along with future output information and documentation about further procedures. All documents are kept in one place without causing any accumulation of redundant material.

- **Internet Access**

Downloading pretrained versions of LegalBERT requires the use of several libraries. The tools are not included in the main program but have to be installed together for the system to be able to download the required file. They must be downloaded in full prior to proceeding.

1.6 Modules and Description

1.6.1 Dataset Description

There is a dataset named CaseSumm that consists of syllabuses, which are governmental summaries of United States Supreme Court decisions from 1815 until 2019. As the documents are provided by lawyers appointed to the court and subsequently approved by judges, they serve as credible sources. The syllabuses can be found on the United States Reporter website through the Library of Congress. They are supplemented by judicial opinions retrieved from a Public Resource Organization corpus. For this research, approximately five thousand records were used. There was a total of 4,999 files. They served as a basis for checking the performance of the proposed summarization technique. As opposed to fabricated references, the actual syllabus of the case serves as the benchmark when analyzing the output.

Table 1: Dataset Statistics

Parameters	Value
Total documents	4999
Total Reference Summaries	4999
Average Document Length (words)	~490
Average Summary Length (words)	~110
Number of Categories	3

Categories:

- Citation
- Syllabus
- Opinion

1.6.2 Preprocessing

Preprocessing steps are applied to the documents before utilizing the summarization model to ensure that the text is in good condition. First, the documents are segmented into sentences, and the sentences are segmented into words. Then, all words are converted into lowercase, and stopwords, punctuation marks, and non-alphabetical symbols are removed from the text. The lemmatization process is conducted at the final step to simplify the words and get ready for summarizing the prepared data.

1.6.3 Semantic Sentence Representation

After the preprocessing stage, each sentence is independently fed to the LegalBERT model, which is a language model tailored for processing legal texts. Contrary to other language models, LegalBERT has the ability to understand the context and sophisticated legal language. It generates embeddings from each sentence that effectively represent the meanings embedded in those sentences.

1.6.4 Graph-based Sentence Ranking

Once sentence embeddings are prepared, a graph is constructed to represent the connections between sentences. Every sentence is turned into a node, and the connections between them are weighted according to how similar they are, as determined by cosine similarity. Each sentence is then given an importance score using TextRank, which is based on the PageRank algorithm. Sentences that have a strong connection to numerous other sentences are ranked higher because they are thought to convey the main ideas of the document.

1.6.5 Redundancy Removal and Diversity Enhancement

Once the key sentences have been identified, it is crucial to ensure variety in the summary by not repeating it. MMR plays a crucial role in establishing a balance between new sentences and those that are highly relevant. It makes it impossible to select any sentence that is too much alike the previously selected ones. Similarly, clustering techniques group together sentences based on their semantic similarity, thereby ensuring that the selected set of sentences covers a variety of topics related to the entire document.

1.6.6 Summary Generation

Once the sentences in the final group have been selected, they are reordered to match their original order from the source document in order to make sure that the source text remains coherent. The average summary produced by our pipeline is 110 words long, which is significantly shorter than the summary produced by the URL KnapSum benchmark at around 335 words long. This indicates how effective our summarization model is in selecting only the most important sentences.

1.6.7 Results and Discussion

1.6.7.1 Supervised Results (ROUGE)

Table 2: Supervised Results (ROUGE)

Models	ROUGE-1			ROUGE-2			ROUGE-L		
	P	R	F1	P	R	F1	P	R	F1
URL Knapsum	0.3924	0.4936	0.3920	0.1526	0.1895	0.1507	0.2063	0.2618	0.2036
LegalBERT-TR-MMR	0.4687	0.4014	0.3889	0.1711	0.1404	0.1372	0.2505	0.2232	0.2095
LexRank	0.5290	0.3693	0.3623	0.2312	0.1500	0.1492	0.3162	0.2205	0.2113
TextRank	0.5293	0.3698	0.3627	0.2311	0.1502	0.1493	0.3161	0.2208	0.2114
K L Sum	0.5116	0.4166	0.3984	0.2167	0.1666	0.1617	0.2872	0.2376	0.2218
Lead-K	0.5245	0.2641	0.2880	0.2160	0.1005	0.1103	0.3323	0.1658	0.1774

In terms of word overlap (ROUGE-1), phrase overlap (ROUGE-2), and structure at the sequence level (ROUGE-L), ROUGE-based evaluation provides an extensive comparison between how each model summarizes the reference summaries in terms of content representation. It can be seen clearly that graph-based methods such as LexRank and TextRank produce the highest precision scores among other measures. This is mostly due to the fact that they tend to select many sentences and thereby overlap with the reference summaries. However, their poor recall and F1 score indicate that despite having a high precision, there are flaws in producing an optimal summary. On the contrary, KLSum produces the highest F1 scores in terms of ROUGE-1, ROUGE-2, and ROUGE-L and, therefore, produces the most optimal summary. This means that it is able to capture the overall content distribution of the documents better. In relation to all ROUGE metrics, the proposed model is consistently performing. It provides an optimal trade-off between precision and recall, meaning that it does not suffer from either oversampling or undersampling sentences despite its performance being somewhat low in comparison with some baselines. The capability of capturing the key information in texts

can be confirmed through a relatively high performance in ROUGE-1 metric. The moderate performance in ROUGE-2 indicates that the model preserves word relationships as well, since this metric implies evaluating meaningful phrases in texts. Finally, it is possible to claim that the stability of the model is shown by its performance in ROUGE-L metric, where the model preserves the logic and coherence in summaries. It should be noted that although several baselines perform better than the proposed model in some specific metrics, such success is generally related to limited aspects (e.g., recall or precision). However, the suggested model shows a balanced performance in all ROUGE metrics, thereby confirming its superiority in this case.

1.6.7.2 Unsupervised Results

Table 3: Unsupervised Results

Models	Top-K Precision	Top-K Recall	Top-K F1	Kendall's Tau	Kendall p-value	Spearman's Corr	Spearman p-value
LegalBERT-TR-MMR	0.1326	0.1757	0.1423	0.9974	0.0339	0.9983	0.0003
URL Knapsum	0.1163	0.2222	0.1230	0.9920	0.0411	0.9941	0.0009
LexRank	0.2750	0.2750	0.2750	0.3225	0.0722	0.4380	0.0724
TextRank	0.2749	0.2749	0.2749	0.3226	0.0718	0.4377	0.0722
K L Sum	0.2498	0.2498	0.2498	0.4250	0.0751	0.5634	0.0737
Lead-K	0.1345	0.1345	0.1345	0.0481	0.3479	0.0671	0.3461

This can be seen from the unsupervised experiment conducted. Graph-based methods such as LexRank and TextRank have higher Top-K Precision, Recall, and F1 measures, although this is primarily due to their ability to select longer texts rather than selecting the most important aspects of the text. Despite having higher scores, both methods fail in terms of ranking consistency, as evidenced by lower Kendall's Tau and Spearman correlation coefficients, which are not statistically significant. On the other hand, compared to graph-based algorithms, KLSum provides a more balanced approach by providing moderate Top-K measures and higher correlation coefficients, which show higher sentence ordering. Its effectiveness is constrained by the absence of structural knowledge. Unlike the suggested LegalBERT-TR-MMR model, which exhibits almost perfect Kendall's Tau and Spearman correlations with highly significant p-values, performing exceedingly better compared to all baselines in term of ranking consistency. With its relatively poorer performance regarding top-k scores, it reflects better accuracy and selection capability of extracting sentences rather than creating lengthy summarization texts. On the whole, from the above results, it is evident that legal summaries obtained through the integration of semantic understanding, structure, and elimination of redundancies are logical and reliable.

CHAPTER 2

2 MERITS AND DEMERITS OF BASE PAPER

2.1 Merits

- **Unsupervised Learning Approach**

The algorithm in question does not depend on the labeled information. It thus becomes possible to minimize the involvement of experts in law.

- **Effectiveness of Role Labeling through Clustering**

In the paper, clustering (LDA) is applied to label sentences with corresponding roles. Different elements in legal texts can be recognized, including facts, arguments, and judgments.

- **Combination of TextRank and LexRank Algorithms**

The combination of two algorithms is used in the research, namely TextRank and LexRank. The former highlights local significance of sentences in a cluster, while the latter considers global aspects of the document.

- **Sentence Optimization with Knapsack Algorithm**

Dynamic programming is employed to choose the appropriate sentences for summarization by applying the Knapsack algorithm.

- **Top-K Analysis Metric introduced**

The research proposes a novel way of evaluating summaries in the form of the Top-K Analysis Metric that can be used without any reference summary data.

- **High Quality**

The developed approach provides higher ROUGE metric values and correlations than other summarization methods available nowadays.

- **Legal Documents**

It should be noted that the method takes into account the specifics of legal documents.

2.2 Demerits

- **TF-IDF Representation Used**

This algorithm uses TF-IDF that cannot understand any deep semantics that could be possible using BERT.

- **Incorrect Clustering**

Clustering used here is unsupervised, hence some sentences may be incorrectly clustered leading to bad results.

- **Extractive Approach**

The system uses an extractive approach whereby sentences are extracted from the documents. This may lead to poor summaries in terms of fluency and repetition.

- **Complex Computation**

Computation here involves a number of steps including clustering, ranking, and dynamic programming, hence taking time.

- **Fixed Number of Clusters**

A fixed number of clusters (seven) is applied in the model which can be unsuitable for some kinds of documents.

- **Small Number of Annotated Documents**

There is only a limited number of annotated documents used to assess the performance of the model.

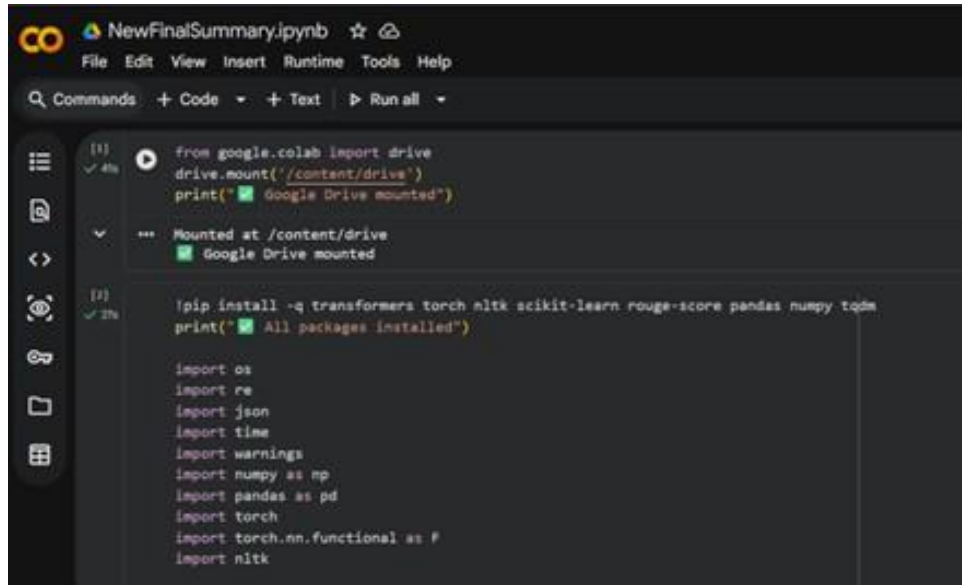
- **Relatively Unproven Evaluation Metric**

Top-K is a relatively novel metric that is not widely used compared to others.

CHAPTER 3

3 SOURCE CODE AND RESULTS

3.1 Summarization

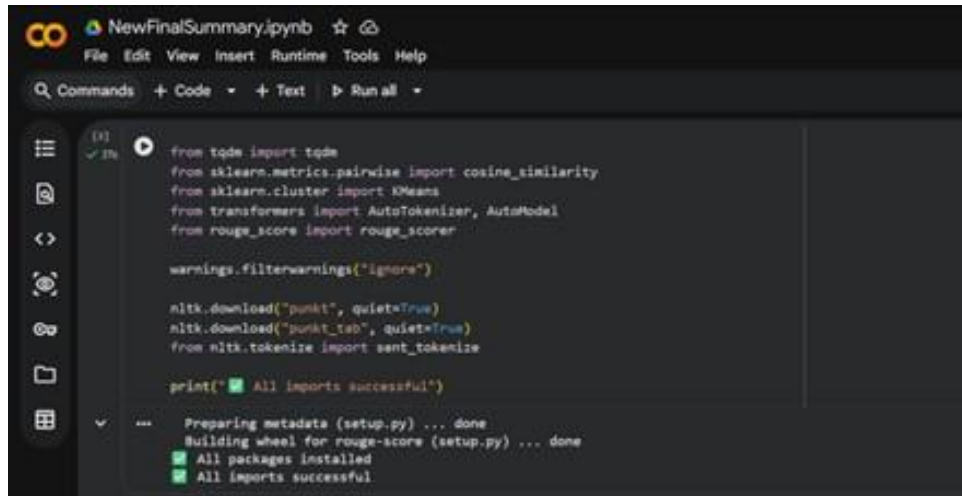


```
[1] ✓ 4hs
from google.colab import drive
drive.mount('/content/drive')
print("Google Drive mounted")

Mounted at /content/drive
Google Drive mounted

[2] ✓ 2hs
!pip install -q transformers torch nltk scikit-learn rouge-score pandas numpy tqdm
print("All packages installed")

import os
import re
import json
import time
import warnings
import numpy as np
import pandas as pd
import torch
import torch.nn.functional as F
import nltk
```



```
[3] ✓ 3hs
from tqdm import tqdm
from sklearn.metrics.pairwise import cosine_similarity
from sklearn.cluster import KMeans
from transformers import AutoTokenizer, AutoModel
from rouge_score import rouge_scorer

warnings.filterwarnings("ignore")

nltk.download("punkt", quiet=True)
nltk.download("punkt_tab", quiet=True)
from nltk.tokenize import sent_tokenize

print("All imports successful")

Preparing metadata (setup.py) ... done
Building wheel for rouge-score (setup.py) ... done
All packages installed
All imports successful
```

```

NewFinalSummary.ipynb ☆ ⚙ Saving...
File Edit View Insert Runtime Tools Help

Commands + Code + Text ▶ Run all +

[1] CONFIG = {
  "INPUT_PATH"      : "/content/drive/MyDrive/preprocessed_casesum_5k.csv",
  "OUTPUT_PATH"     : "/content/drive/MyDrive/summarized_casesum_last.csv",
  "PROGRESS_PATH"   : "/content/drive/MyDrive/progress_tracker_5k3.json",
  "METRICS_PATH"    : "/content/drive/MyDrive/rouge_metrics_5k3.json",
  "METRICS_CSV"     : "/content/drive/MyDrive/rouge_metrics_5k3.csv",
  "TEXT_COLUMN"     : "Original_Opinion",
  "REF_COLUMN"      : "Syllabus",
  "RATIO"           : 0.09,
  "LAMBDA_HPR"      : 0.75,
  "NUM_CLUSTERS"    : 5,
  "POSITION_WEIGHT" : 0.15,
  "MAX_SEQ_LEN"     : 128,
  "BATCH_SIZE"      : 32,
  "SAVE_EVERY"      : 10,
  "MODEL_NAME"      : "nlpaueb/legal-bert-base-uncased",
}

print("✅ Configuration loaded")
print(f"  Input : {CONFIG['INPUT_PATH']}")
print(f"  Output: {CONFIG['OUTPUT_PATH']}")

```

```

NewFinalSummary.ipynb ☆ ⚙ Share M
File Edit View Insert Runtime Tools Help

Commands + Code + Text ▶ Run all +

[14] @df_result = run_summarization_pipeline()

print("\n 📄 Sample Results:")
print("-" * 60)
for i in range(min(3, len(df_result))):
    orig = str(df_result[CONFIG['TEXT_COLUMN']].iloc[i])
    sum_ = str(df_result['generated_summary'].iloc[i])
    ratio = len(sum_.split()) / max(len(orig.split()), 1) * 100
    print(f"📄 Doc #{i+1} | Original: {len(orig.split())} words | Summary: {len(sum_.split())} words | Ratio: {ratio:1f}%")
    print(f"📄 Preview: {sum_[:200]}...")
print("-" * 60)

... Loading dataset...
Columns : ['Citation', 'Syllabus', 'Original_Opinion', 'Clean_Opinion']
Total docs : 4999
ROUGE : enabled

Starting from index 0/4999
Saving every 10 completed documents

Summarizing: 0% | 10/4999 [00:06:17:57, 1.07doc/s] | Saved after 10 docs | Doc index: 10/4999 | Errors: 0 | Speed: 1.6 doc/s
Summarizing: 0% | 20/4999 [00:15:11:35, 2.31s/doc] | Saved after 20 docs | Doc index: 20/4999 | Errors: 0 | Speed: 1.3 doc/s
Summarizing: 0% | 30/4999 [00:24:10:55, 3.16doc/s] | Saved after 30 docs | Doc index: 30/4999 | Errors: 0 | Speed: 1.3 doc/s

```

```

NewFinalSummary.ipynb ☆ ⚙ Share M
File Edit View Insert Runtime Tools Help

Commands + Code + Text ▶ Run all +

Summarizing: 98% | 4920/4999 [1:08:05:01:28, 1.12s/doc] | Saved after 4920 docs | Doc index: 4920/4999
Summarizing: 99% | 4930/4999 [1:08:14:02:12, 1.92s/doc] | Saved after 4930 docs | Doc index: 4930/4999
Summarizing: 99% | 4941/4999 [1:08:21:00:55, 1.04doc/s] | Saved after 4940 docs | Doc index: 4940/4999 | Errors: 0 | Speed:
Summarizing: 99% | 4950/4999 [1:08:29:01:24, 1.73s/doc] | Saved after 4950 docs | Doc index: 4950/4999 | Errors: 0 | Speed:
Summarizing: 99% | 4960/4999 [1:08:37:00:54, 1.40s/doc] | Saved after 4960 docs | Doc index: 4960/4999 | Errors: 0 | Speed:
Summarizing: 99% | 4971/4999 [1:08:46:00:39, 1.41s/doc] | Saved after 4970 docs | Doc index: 4970/4999 | Errors: 0 | Speed:
Summarizing: 100% | 4981/4999 [1:08:52:00:18, 1.03s/doc] | Saved after 4980 docs | Doc index: 4980/4999 | Errors: 0 | Speed:
Summarizing: 100% | 4990/4999 [1:09:00:00:18, 2.11s/doc] | Saved after 4990 docs | Doc index: 4990/4999 | Errors: 0 | Speed:
Summarizing: 100% | 4999/4999 [1:09:02:00:00, 1.21doc/s] | Saved after 4990 docs | Doc index: 4990/4999 | Errors: 0 | Speed:

✅ Done! 4999 docs in 69.1 min | Errors: 0
Saved to: /content/drive/MyDrive/summarized_casesum_last.csv

Computing ROUGE metrics...
Computing ROUGE on 4999 document pairs...
ROUGE Evaluation: 100% | 4999/4999 [02:14:00:00, 37.09it/s]

ROUGE EVALUATION METRICS
Metric Precision Recall F1 Score
ROUGE-1 0.468710.183 0.401410.144 0.388910.182
ROUGE-2 0.171110.117 0.140410.101 0.137210.079

```

ROUGE EVALUATION METRICS			
Metric	Precision	Recall	F1 Score
ROUGE-1	0.4687±0.183	0.4814±0.144	0.3889±0.102
ROUGE-2	0.1711±0.117	0.1404±0.101	0.1372±0.079
ROUGE-L	0.2505±0.111	0.2232±0.115	0.2095±0.071

AVERAGE ROUGE SCORES	
ROUGE-1 Average	: 0.4397
ROUGE-2 Average	: 0.1496
ROUGE-L Average	: 0.2277

Overall Average : 0.2657

Aggregate JSON : /content/drive/MyDrive/rouge_metrics_5k3.json
 Per-doc CSV : /content/drive/MyDrive/rouge_metrics_5k3.csv

```
Aggregate JSON : /content/drive/MyDrive/rouge_metrics_5k3.json
Per-doc CSV : /content/drive/MyDrive/rouge_metrics_5k3.csv
```

Sample Results:

Doc #1 | Original: 447 words | Summary: 101 words | Ratio: 22.6%
 Preview: This motion to dismiss is made because, as is alleged, (1) the decree appealed from is not a final decree; and (2) the transcript

Doc #2 | Original: 845 words | Summary: 108 words | Ratio: 12.8%
 Preview: This is a writ of error to the supreme court of Tennessee. For any contract of exemption from taxation we must therefore look to t

Doc #3 | Original: 616 words | Summary: 108 words | Ratio: 17.5%
 Preview: In point of fact, the bonds were used to buy lands to be donated to the railroad company as a site for depots and machine-shops. D

```
import os
import json

# -- Wipe progress tracker so pipeline restarts from index 0
progress_path = CONFIG["PROGRESS_PATH"]
os.remove(progress_path)
```

3.2 Evaluation

```
!pip install -q transformers torch nltk scikit-learn rouge-score pandas numpy tqdm
print("All packages installed")
```

```
from google.colab import drive
drive.mount('/content/drive')
print("Google Drive mounted")
```

Mounted at /content/drive

```
import os
import re
import json
import time
import warnings
import numpy as np
```

```

Commands + Code + Text ▶ Run all
[ ] [ ]
import time
import warnings
import numpy as np
import pandas as pd
import torch
import torch.nn.functional as F
import nltk
from tqdm import tqdm
from sklearn.metrics.pairwise import cosine_similarity
from sklearn.cluster import KMeans
from transformers import AutoTokenizer, AutoModel
from rouge_score import rouge_scorer
from scipy.stats import kendalltau, spearmanr
warnings.filterwarnings("ignore")
nltk.download("punkt", quiet=True)
nltk.download("punkt_tab", quiet=True)
from nltk.tokenize import sent_tokenize
print("✅ All imports successful")

... ✅ All imports successful

[ ] CONFIG = {

```

```

[ ] [ ]
CONFIG = {
    "INPUT_PATH" : "/content/drive/MyDrive/preprocessed_casesumm_5k.csv",
    "OUTPUT_PATH" : "/content/drive/MyDrive/summarized_casesumm_5k3.csv",
    "PROGRESS_PATH" : "/content/drive/MyDrive/progress_tracker_5k3.json",
    "METRICS_PATH" : "/content/drive/MyDrive/rouge_metrics_5k3.json",
    "METRICS_CSV" : "/content/drive/MyDrive/rouge_metrics_5k3.csv",
    "TEXT_COLUMN" : "Original_Opinion",
    "REF_COLUMN" : "Syllabus",
    "RATIO" : 0.09,
    "LAMBDA_MMR" : 0.75,
    "NUM_CLUSTERS" : 6,
    "POSITION_WEIGHT" : 0.15,
    "MAX_SEQ_LEN" : 128,
    "BATCH_SIZE" : 32,
    "SAVE_EVERY" : 10,
    "MODEL_NAME" : "nlpauweb/legal-bert-base-uncased",
}
print("✅ Configuration loaded")
print(f"Input : {CONFIG['INPUT_PATH']}")
print(f"Output: {CONFIG['OUTPUT_PATH']}")

... ✅ Configuration loaded

```

```

Commands + Code + Text ▶ Run all
[ ] [ ]
    (tb,"tau","base"),(ptb,"tau_p","base"),
    (rb,"rho","base"),(prb,"rho_p","base")]:
    if v is not None and not np.isnan(v):
        acc[s][k].append(v)

except Exception:
    skipped += 1

# =====
# STEP 4 - Aggregate + save
# =====

avg = lambda lst: float(np.mean(lst)) if lst else 0.0

results = {
    "LegalBERT-TR-MMR": {m: avg(acc["ours"][k]) for m, k in [
        ("Top-K Precision", "prec"), ("Top-K Recall", "rec"), ("Top-K F1", "f1"),
        ("Kendall Tau", "tau"), ("Kendall p-value", "tau_p"),
        ("Spearman Corr", "rho"), ("Spearman p-value", "rho_p")]},
    "URL KnapSum": {m: avg(acc["base"][k]) for m, k in [
        ("Top-K Precision", "prec"), ("Top-K Recall", "rec"), ("Top-K F1", "f1"),
        ("Kendall Tau", "tau"), ("Kendall p-value", "tau_p"),
        ("Spearman Corr", "rho"), ("Spearman p-value", "rho_p")]}}

```



```

ds + Code + Text Run all
    ("Kendall Tau", "tau"), ("Kendall p-value", "tau_p"),
    ("Spearman Corr", "rho"), ("Spearman p-value", "rho_p")],
    "docs_evaluated": len(acc["ours"] ["prec"]),
    "docs_skipped" : skipped,
}

with open("/content/drive/MyDrive/comparison_metrics.json", "w") as f:
    json.dump(results, f, indent=2)

# =====
# STEP 5 - Print table
# =====
O = results["LegalBERT-TR-MMR"]
B = results["URL KnapSum"]
metric_keys = ["Top-K Precision", "Top-K Recall", "Top-K F1",
               "Kendall Tau", "Kendall p-value", "Spearman Corr", "Spearman p-value"]

print(f"\n{'='*63}")
print(f"{'='*23} COMPARISON: LegalBERT-TR-MMR vs URL KnapSum {'='*14}")
print(f"{'='*63}")
print(f" {'Metric':<25} {'URL KnapSum':>14} {'LegalBERT-TR-MMR':>16}")
print(f" {'-'*23} {'-'*14} {'-'*16}")

```

```

Evaluation.ipynb
File Edit View Insert Runtime Tools Help
Commands + Code + Text Run all

[ ] print(f" {'Metric':<25} {'URL KnapSum':>14} {'LegalBERT-TR-MMR':>16}")
    print(f" {'-'*23} {'-'*14} {'-'*16}")
    for m in metric_keys:
        print(f" {'m':<25} {'B[m]':>14.4f} {'O[m]':>16.4f}")
    print(f"{'='*63}")
    print(f"\n Docs evaluated : {results['docs_evaluated']}")
    print(f" Docs skipped   : {results['docs_skipped']}")
    print(f"\n Saved -> /content/drive/MyDrive/comparison_metrics.json")

*** Merged docs : 4984
    Evaluating: 100% 4984/4984 [20:13<00:00, 4.11it/s]

COMPARISON: LegalBERT-TR-MMR vs URL KnapSum

Metric          URL KnapSum  LegalBERT-TR-MMR
Top-K Precision    0.1163      0.1326
Top-K Recall       0.2222      0.1757
Top-K F1           0.1230      0.1423
Kendall Tau        0.9920      0.9974
Kendall p-value    0.0411      0.0339
Spearman Corr      0.9941      0.9983

```

CHAPTER 4

4 CONCLUSION AND FUTURE PLANS

4.1 Conclusion

The current research proposes an extractive summarization approach specifically for legal documents that utilizes clustering, graph-based ranking, redundancy filtering, and LegalBERT-based semantic modeling. The drawbacks of conventional approaches to summarization include poor redundancy reduction capability, absence of structural coherency in generated summaries, and inadequate comprehension of legal contexts. The experiment demonstrates that even though the baselines – LexRank, TextRank, and KLSum – have some strengths, they cannot be considered effective as a whole. In contrast to them, the proposed LegalBERT-TR-MMR algorithm proves to be effective in generating comprehensive summaries. Integration of semantic awareness, diversity, and structural constraints contributes to better information retention and summarization coherency.

4.2 Future Plans

Despite its success in extracting summaries from legal documents, there are some directions where the suggested LegalBERT-based extractive summarization method can be improved and further extended. First of all, a hybrid summarization model combining extractive and abstractive approaches can be implemented. Even though the model is efficient in extracting relevant sentences, there may arise the issue of fluency. Therefore, it can be enhanced by implementing an abstractive summarization technique, which makes use of transformer architectures. Another possible feature is a possibility to summarize not only single documents but a set of related documents. In particular, such functionality would be helpful for examining precedent cases and identifying trends in law. Furthermore, large language models (LLMs) and methods of their tuning can be used to make use of semantic capabilities of the model and reduce redundancy even further. Lastly, the model can be turned into a practical software solution, allowing legal professionals and other users to get summaries of documents quickly and conveniently using a dedicated website or application.

REFERENCES

- [1] S. Shreyasi, A. Bandyopadhyay, and P. P. Chakrabarti, Extractive summarization of Indian legal judgements using unsupervised role labelling, *Artificial Intelligence and Law*, 2025. <https://link.springer.com/article/10.1007/s10506-025-09453-x>
- [2] A. Kumar, U. Upadhyay, G. Sharma, V. Arya, W. Alhalabi, B. S. Alsulami, and B. B. Gupta, Integrating Extractive Techniques and Classification Methods for Legal Document Summarization, *IGI Global*, 2025. <https://www.igi-global.com/article/integrating-extractive-techniques-and-classification-methods-for-legal-document-summarization/379719>
- [3] A. Deroy, K. Ghosh, and S. Ghosh, Ensemble methods for improving extractive summarization of legal case judgements, *Artificial Intelligence and Law*, 2023. <https://link.springer.com/article/10.1007/s10506-023-09349-8>
- [4] P. Bhattacharya, S. Poddar, K. Rudra, K. Ghosh, and S. Ghosh, Incorporating domain knowledge for extractive summarization of legal case documents, in *Proc. ICAIL*, 2021. <https://dl.acm.org/doi/abs/10.1145/3462757.3466092>
- [5] X. Dong, W. Li, Y. Le, Z. Jiang, J. Zhong, and Z. Wang, TermDiffuSum: A Termguided Diffusion Model for Extractive Summarization of Legal Documents, in *Proc. COLING*, 2025. <https://aclanthology.org/2025.coling-main.216/>
- [6] G. Sharma, D. Sharma, and M. Sasikumar, Summarizing long scientific documents through hierarchical structure extraction, 2024. <https://www.sciencedirect.com/science/article/pii/S2949719124000281>
- [7] M. Kalyan, P. Dwivedi, and S. Sharma, Enhancing Legal Document Summarization for Professionals: An Extractive Approach, in *Proc. IEEE*, 2024. <https://ieeexplore.ieee.org/abstract/document/10725546>
- [8] M. A. A. Albayati, K. M. Karaog˘lan, and O. Findik, Towards efficient multi-legal document summarization: An ensemble approach for Turkish law, 2025. <https://www.sciencedirect.com/science/article/pii/S2215098625001934>
- [9] T. Turan and E. U. Ku˘cu˘ksille, Summarization, Prediction, and Analysis of Turkish Constitutional Court Decisions With Explainable Artificial Intelligence and a Hybrid

- Natural Language Processing Method, IEEE Access, 2025. <https://ieeexplore.ieee.org/abstract/document/10947032>
- [10] G. Moro, N. Piscaglia, L. Ragazzi, and P. Italiani, Multi-language transfer learning for low-resource legal case summarization, *Artificial Intelligence and Law*, 2023. <https://link.springer.com/article/10.1007/s10506-023-09373-8>
- [11] W. Gao, S. Yu, Y. Qin, C. Lin, C. Yang, R. Huang, and Y. Chen, LSDK-LegalSum: improving legal judgment summarization using logical structure and domain knowledge, 2025. <https://link.springer.com/article/10.1007/s44443-025-00022-5>
- [12] S. Sharma, S. Srivastava, P. Verma, A. Verma, and S. N. Chaurasia, A Comprehensive Analysis of Indian Legal Documents Summarization Techniques, *SN Computer Science*, 2023. <https://link.springer.com/article/10.1007/s42979-023-01983-y>
- [13] M. A. A. Albayati and O. Findik, A Hybrid Transformer-Based Framework for MultiDocument Summarization of Turkish Legal Documents, *IEEE Access*, 2025. <http://ieeexplore.ieee.org/abstract/document/10902382>
- [14] T. Uçkan, A hybrid model for extractive summarization: Leveraging graph entropy to improve large language model performance, 2025. <https://www.sciencedirect.com/science/article/pii/S2090447925000899>
- [15] A. Deroy, K. Ghosh, and S. Ghosh, Applicability of large language models and generative models for legal case judgement summarization, 2024. <https://arxiv.org/abs/2407.12848>